# Aportes de los cinco integrantes en todos los temas

## SIGHA · Sistema de Gestión de Horarios Académicos

Programación Orientada a Objetos · Periodo académico 2026-2

Nuestro grupo ha trabajado en conjunto en la definición de SIGHA y su UML inicial. Para organizar la participación individual, planteamos ideas de cada integrante en todos los temas estudiados, desde los fundamentos de Python hasta abstracción y polimorfismo.

Reunimos 30 propuestas: seis por integrante. Se relacionan con nuestro informe y diagrama, y orientan el desarrollo posterior. El avance actual sigue siendo de definición y diseño; las propuestas no se presentan como funciones ya implementadas.

### Temas que trabajaremos entre todos

Cada integrante participa en los seis temas. Las ideas pueden complementarse: una persona puede proponer una base común y otra precisar una especialización, una validación o una forma de comprobarla.

| Tema | Aplicación al proyecto |
|---|---|
| Fundamentos de Python | Organizar días, periodos y asignaciones con estructuras básicas. |
| Clases objetos y métodos | Precisar recursos, constructores, atributos y operaciones. |
| Encapsulamiento | Controlar cambios y validar los datos del modelo. |
| Herencia | Reutilizar datos y comportamientos que tengan una relación coherente. |
| Abstracción | Definir operaciones y responsabilidades comunes. |
| Polimorfismo | Usar una operación común con comportamientos distintos. |

### Integrantes

Ian Rafael Avila Delgado

Jeremy Jair Eduarte Lopez

Jose Gabriel Intriago Cedeño

Anahi Carolina Robayo Tuarez

Heiko Marcelo Vera Ponce

# Propuestas de Ian Rafael Avila Delgado

Seis ideas para relacionar lo aprendido con nuestro diseño de SIGHA.

### 1 Fundamentos de Python · Semana como matriz

Propongo representar la consulta semanal mediante una lista de listas: las filas corresponderían a periodos y las columnas a días. Recorrería esa estructura con bucles para mostrar las asignaturas y los bloques libres de un curso. Comprobaría que cada clase aparezca en la posición que corresponda a su día y periodo.

### 2 Clases objetos y métodos · Horario relacionado con sus recursos

Propongo precisar el constructor de Horario para que reciba objetos de Asignatura, Docente, Aula, Curso y Periodo. Así desarrollaríamos las relaciones de nuestro UML en lugar de repetir sus datos en textos aislados. Comprobaría que consultar el detalle del horario permita identificar correctamente todos sus recursos.

### 3 Encapsulamiento · Horas protegidas y actualización controlada

Propongo que Periodo mantenga controladas sus horas y permita consultarlas mediante métodos o propiedades. Antes de actualizarlas, revisaría que el fin sea posterior al inicio. Comprobaría que un cambio inválido no modifique el periodo que ya era correcto, para proteger las asignaciones que lo utilizan.

### 4 Herencia · Datos comunes en Persona

Propongo reunir nombre e identificación en Persona y utilizarla como base de Docente y Estudiante. Con ello evitaríamos repetir su inicialización. Revisaría los atributos comunes y plantearía esta generalización como una actualización de nuestro UML, que actualmente presenta a ambas entidades por separado.

### 5 Abstracción · Contrato para las consultas

Propongo definir ConsultaHorario como una clase abstracta que requiera el método consultar(). Esta base indicaría qué operación deben ofrecer las consultas concretas. Antes de implementarla, acordaría sus parámetros y su resultado: una colección de horarios. La base no se utilizaría para crear consultas sin un criterio concreto.

### 6 Polimorfismo · Una llamada para distintos criterios

Propongo utilizar consultar() tanto con una ConsultaPorCurso como con una ConsultaPorDocente. Cada objeto aplicaría su criterio y respondería con las asignaciones correspondientes. Comprobaría el comportamiento con un curso y un docente que tengan clases diferentes; la llamada sería la misma, pero sus resultados podrían ser distintos.

# Propuestas de Jeremy Jair Eduarte Lopez

Seis ideas para relacionar lo aprendido con nuestro diseño de SIGHA.

### 1 Fundamentos de Python · Funciones para revisar las entradas

Propongo separar en funciones la lectura y revisión de datos básicos, como el número de periodo y la capacidad de un aula. Utilizaría conversiones y condicionales para aceptar valores coherentes. Probaría entradas válidas, valores vacíos y números fuera del rango previsto antes de incorporarlos a los recursos del sistema.

### 2 Clases objetos y métodos · Asignaturas con atributos y operaciones claras

Propongo detallar Asignatura con nombre, código y horas semanales, de acuerdo con el informe. Su constructor inicializaría cada objeto y sus métodos permitirían consultar sus datos. Crearía dos asignaturas de prueba para comprobar que sus valores son individuales y no se confunden con atributos compartidos de la clase.

### 3 Encapsulamiento · Carga semanal con valores válidos

Propongo controlar la modificación de las horas semanales de una asignatura mediante una operación específica. Esta operación verificaría que el nuevo valor sea un entero positivo antes de cambiarlo. Comprobaría que se rechacen valores negativos o nulos y que, después del rechazo, se conserve el valor anterior.

### 4 Herencia · Especialización de un aula

Propongo estudiar LaboratorioTI como una especialización de Aula que añada cantidad de equipos. Su constructor reutilizaría número y capacidad mediante super().__init__(). Antes de incorporarla al UML, revisaríamos si la planificación necesita distinguir ese tipo de espacio. La especialización ampliaría Aula, sin reemplazar los datos comunes.

### 5 Abstracción · Contrato para validar asignaciones

Propongo una base abstracta ValidadorAsignacion con una operación validar(candidato, horarios). Su responsabilidad sería definir qué recibe y qué devuelve una validación, sin resolver todos los conflictos en la clase base. Las implementaciones concretas revisarían docente, aula o curso siguiendo las reglas de Horario y Periodo.

### 6 Polimorfismo · Validaciones intercambiables

Propongo que ValidadorDocente, ValidadorAula y ValidadorCurso implementen la misma operación validar(). Un proceso podría recorrer esos objetos y recoger sus resultados sin cambiar la llamada para cada tipo. Comprobaría tres casos separados, cada uno con un único recurso en conflicto, para distinguir el comportamiento de las implementaciones.

# Propuestas de Jose Gabriel Intriago Cedeño

Seis ideas para relacionar lo aprendido con nuestro diseño de SIGHA.

### 1 Fundamentos de Python · Recorrer las asignaciones para buscar cruces

Propongo recorrer las asignaciones existentes y comparar cada una con la nueva propuesta. Con bucles y condicionales revisaría día, horas y recursos coincidentes. Prepararía casos de horas superpuestas, bloques consecutivos y días diferentes. Esta lógica básica ayudaría a desarrollar detectarConflicto() y seSuperponeCon(), que ya aparecen en nuestro UML.

### 2 Clases objetos y métodos · Docentes con disponibilidad individual

Propongo representar a cada docente mediante un objeto con nombre, identificación y disponibilidad. Definiría métodos para consultar y actualizar esa información. Utilizaría dos docentes de ejemplo para comprobar que modificar la disponibilidad de uno no altera la del otro, evitando tratar la disponibilidad como un dato compartido de toda la clase.

### 3 Encapsulamiento · Cambios de disponibilidad con revisión previa

Propongo controlar la actualización de disponibilidad del docente mediante un método. Primero acordaríamos un formato de días y horas, y después validaríamos los datos antes de sustituir los anteriores. También propondría revisar las asignaciones afectadas por el cambio. Comprobaría que un formato inválido no borre la disponibilidad que ya estaba registrada.

### 4 Herencia · Docente reutiliza la base Persona

Propongo que Docente herede los datos comunes de Persona y mantenga su disponibilidad como un atributo propio. Utilizaría super() para inicializar nombre e identificación y un método específico para describir su función académica. Comprobaría que el objeto conserve los datos heredados y los que corresponden únicamente al docente.

### 5 Abstracción · Operación común para consultar disponibilidad

Propongo definir un contrato de disponibilidad con una operación esta_ocupado(dia, periodo, horarios). Docente, Aula y Curso podrían ofrecer esa operación usando sus asignaciones. Acordaría primero los datos de entrada y el resultado esperado. Esta propuesta amplía el diseño y deberá revisarse antes de introducir nuevas abstracciones en el UML.

### 6 Polimorfismo · Disponibilidad según el recurso consultado

Propongo utilizar esta_ocupado() con distintos recursos. El docente buscaría sus clases, el aula revisaría su ocupación y el curso comprobaría sus asignaturas. Prepararía casos donde solo uno esté ocupado. La operación común permitiría obtener respuestas diferentes según el objeto, en lugar de resolver todos los casos con una única cadena de condicionales.

# Propuestas de Anahi Carolina Robayo Tuarez

Seis ideas para relacionar lo aprendido con nuestro diseño de SIGHA.

### 1 Fundamentos de Python · Selección y comprobación de días

Propongo utilizar una lista de días académicos y una función para convertir una selección numérica en su día correspondiente. Los condicionales revisarían el rango permitido y un bucle mostraría las opciones. Probaría selecciones válidas e inválidas para evitar que se construyan horarios con un día que no esté contemplado.

### 2 Clases objetos y métodos · Estudiantes relacionados con su curso

Propongo precisar los objetos Estudiante con los datos del UML y su relación con Curso. La consulta del estudiante obtendría el horario del curso o paralelo al que pertenece. Comprobaría que dos estudiantes del mismo grupo consulten la misma planificación, sin registrar por duplicado cada asignación de clase.

### 3 Encapsulamiento · Modificación segura de un horario

Propongo que modificarAsignacion() revise una propuesta completa antes de cambiar el horario existente. Primero se validarían día, periodo y recursos; después se comprobarían los cruces. Si la propuesta no es válida, conservaríamos la asignación anterior. Probaría un cambio de aula aceptado y otro rechazado por ocupación.

### 4 Herencia · Estudiante como tipo de Persona

Propongo que Estudiante herede de Persona y añada sus datos específicos y su relación con el curso. Revisaría qué métodos puede reutilizar y cuáles necesita definir de otra forma. Evitaría hacerlo heredar de Docente, porque esa relación no representa nuestro proyecto. Esta mejora tendría que reflejarse en una revisión del UML.

### 5 Abstracción · Persona con una operación requerida

Propongo estudiar Persona como clase abstracta y exigir una operación consultar_horario(horarios) para sus clases concretas. La base reuniría los datos comunes, mientras Docente y Estudiante completarían la consulta que les corresponde. Con ABC y abstractmethod se evitaría crear una Persona genérica o una hija que deje esa operación pendiente.

### 6 Polimorfismo · Consulta según el tipo de persona

Propongo llamar consultar_horario() tanto sobre un docente como sobre un estudiante. El docente obtendría sus clases y el estudiante las de su curso. Comprobaría ambos resultados con la misma colección de horarios. Cada clase implementaría su propio comportamiento, siguiendo la operación común definida para las personas.

# Propuestas de Heiko Marcelo Vera Ponce

Seis ideas para relacionar lo aprendido con nuestro diseño de SIGHA.

### 1 Fundamentos de Python · Cálculo de horas planificadas

Propongo una función que recorra las asignaciones de una asignatura y acumule su duración. El resultado permitiría comparar las horas programadas con la carga semanal prevista. Utilizaría números, funciones, bucles y condicionales. Probaría una asignatura sin bloques y otra con varios periodos, considerando sus duraciones reales para evitar contar solo la cantidad de clases.

### 2 Clases objetos y métodos · Periodo con inicialización consistente

Propongo detallar Periodo con número, horaInicio y horaFin, de acuerdo con el UML. Un método de instancia permitiría consultar su duración. También estudiaría un método de clase para construir un periodo desde un formato de texto acordado y una operación estática para revisar ese formato, diferenciando self, cls y operaciones sin estado de instancia.

### 3 Encapsulamiento · Lista de estudiantes controlada por Curso

Propongo que Curso controle su colección de estudiantes y exponga operaciones para agregar, retirar y consultar sus integrantes. La consulta devolvería una copia o una vista que no permita alterar directamente la colección interna. Comprobaría que no se añada dos veces al mismo estudiante y que una consulta no modifique el curso accidentalmente.

### 4 Herencia · Sobrescritura del detalle de un aula

Propongo que un laboratorio pueda ampliar el método que presenta los datos de Aula. La versión especializada llamaría a super() para reutilizar número, capacidad y tipo, y añadiría la información de equipos. Revisaría que no se duplique la parte común. Esta idea complementa la propuesta de especialización del aula y no obliga a añadir otra clase diferente.

### 5 Abstracción · Resultados definidos para una consulta abstracta

Propongo completar el contrato de ConsultaHorario acordando sus parámetros y el formato de sus resultados. La operación abstracta exigiría consultar(), y las clases concretas devolverían una colección de horarios, incluso si está vacía. Así definiríamos qué puede esperar el resto del sistema antes de decidir cómo se realiza cada búsqueda.

### 6 Polimorfismo · Servicio que utiliza consultas concretas

Propongo un ServicioConsulta que reciba un objeto de consulta y utilice su método consultar(), sin decidir internamente si es por docente, curso o aula. Podríamos cambiar el criterio sustituyendo ese objeto. Comprobaría el servicio con consultas diferentes y con un recurso sin clases. La propuesta combina polimorfismo y composición, como en la estructura estudiada con las pasarelas.

# Cómo integraremos las propuestas del equipo

### Revisión por tema

Revisaremos las cinco propuestas de cada tema antes de elegir una implementación. Por ejemplo, al trabajar herencia compararemos la base Persona, las responsabilidades de Docente y Estudiante y la posible especialización de Aula. Así conservaremos un diseño coherente, en lugar de crear versiones independientes de las mismas clases.

### De la idea a una decisión compartida

Para cada propuesta acordaremos qué problema resuelve, con qué entidades se relaciona y qué cambio necesita el UML. Si dos ideas describen la misma operación, definiremos una sola firma de método y un resultado esperado. Si no aportan una mejora clara, podremos simplificarlas o dejarlas para otra etapa.

### Abstracción y polimorfismo

Las ideas de consultas, validadores y disponibilidad exploran distintas aplicaciones de estos conceptos. No necesitamos introducir todas esas abstracciones a la vez. Primero podremos elegir una, comprobar que sus implementaciones tienen la misma operación y resultados compatibles, y ampliar el diseño cuando sea necesario.

### Comprobación futura

Antes de implementar cada cambio prepararemos un ejemplo válido, un caso límite y un caso que deba rechazarse. En horarios revisaremos superposiciones, bloques consecutivos y días diferentes. En las consultas revisaremos los resultados del recurso elegido y el caso de una colección vacía.

### Participación individual y avance conjunto

Cada integrante podrá explicar sus seis propuestas y desarrollar los cambios que el grupo acuerde. La contribución individual quedará reflejada en el trabajo real que incorpore, revise y compruebe. Las decisiones sobre el modelo, los nombres de métodos y las reglas del sistema serán compartidas.

### Estado actual y alcance

Contamos con el informe y el UML inicial. Persona, LaboratorioTI y los contratos abstractos son posibles ampliaciones del diseño, que deberán revisarse antes de adoptarse. Este paquete documental no incluye página, implementación ni prácticas personales. Pagos, calificaciones, asistencia y matrícula administrativa permanecen fuera del alcance de SIGHA.